chapter 1 △单位换算：2进制：bit(位)最小单位，字节(byte)B

1B=8bit. $B \xrightarrow{2^{10}} KiB \xrightarrow{2^{10}} MiB \xrightarrow{2^{10}} GiB \xrightarrow{2^{10}} TiB$

10进制：$B \xrightarrow{10^3} KB \xrightarrow{10^3} MB \xrightarrow{10^3} GB \xrightarrow{10^3} TB$

硬件 系统软件 应用软件 system software 包含：操作系统(OS)、编译器compiler …

*语言转化：高级语言(C, Java, C++, VB) High-level $\xrightarrow{compiler}$ 汇编语言 Assembly $\xrightarrow{Assembler}$ 机器语言 machine

product 乘积 quotient 商 reminder 余数

杂：CPU (central Processor Unit) 话缺 max

CPU { Datapath … perform operations on Data; control … 时序控制 (sequence) datapath、memory; cache memory (SRAM) }

Moore's law: 18~24月，↑1倍

性能相关概念：① Response Time 响应时间（完成一项任务所需 time）

② Throughput 吞吐量（单位 time 完成任务总量）

③ Relative Performance：Performance = $\frac{1}{\text{Execution time}}$，speedup = $\frac{(\text{CPU time})_{old}}{(\text{CPU time})_{new}}$

如何↑ILP指令级并行性：{ 加深流水线 (Deeper); 多发机制 (multiple issue)：静态(compiler)、动态(CPU) }

重要公式：① CPU Time = clock cycles × clock cycle time (execution time)

= CPU clock cycles / clock rate

CPU时间 = 时钟周期数 × 时钟周期长度.

② Clock cycles = instruction count × Cycles per instruction

时钟周期数 = 指令数 × CPI（每个指令平均时钟周期数）

③ 功耗 Power = 频率 × 负载电容 × 电压$^2$

④ MIPS（每秒百万条指令）：$MIPS = \frac{\text{instrution count}}{\text{Execution time} \times 10^6} = \frac{\text{clock rate}}{CPI \times 10^6}$

$T_{improved} = T_{affected}/a + T_{unaffected}$

补：pitfall：Amdahl's law，speedup = $\frac{1}{1-f+f/a}$（f为某part运行时间比，a为该part性能个倍数）

power wall（功耗墙）：无法再↓电压，无法散更多热

△注：CPI可用于比较同一ISA的不同实现，不能用于不同ISA比较.

chapter 2. RISC-V为1种ISA，RV32、RV64都有32个寄存器（64位…）

RV32中：$x_0$ ~ $x_{31}$ 32位 register

memory：8位 ← address 32位

☆32位指令长度

PC=PC+4 4的倍数

☆字节地址含义：lw x5, 40(x6)，偏移40字节.

大端序（低字节高地址）、小端序（低字节低地址）.

举例：① 从register 存 memory 中 (store)

0X12345678（高字节）（低字节）

若基址为x100

大端：

| | |
|---|---|
| X103 | 0X78 |
| X102 | 0X56 |
| X101 | 0X34 |
| X100 | 0X12 |

小端：

| | |
|---|---|
| X103 | 0X12 |
| X102 | 0X34 |
| X101 | 0X56 |
| X100 | 0X78 |

memory

大端 load → register 0X12345678

小端 load → 0X78563412

RISCV大端 MIPS、ARM

register 规则：

$x_0$：常值0

$x_1$ (ra)：返回地址

$x_2$ (sp)：栈顶指针

$x_3$ (gp)：global pointer

$x_4$ (tp)：thread pointer

$x_5$~$x_7$, $x_{28}$~$x_{31}$ (t0~t6)：临时变量

$x_8$~$x_9$, $x_{18}$~$x_{27}$ (s0~s11)：saved register

$x_{10}$~$x_{17}$ (a0~a7)：参数信息. reg a0, a1 可作返回值

其中：$x_5$~$x_7$, $x_{10}$~$x_{17}$, $x_{28}$~$x_{31}$ 调用函数不用保留.

一些汇编指令含义：

☆ lw：memory → register 读操作

sw：register → memory 写操作

△对于寄存器，lw 1读1写 sw 2读0写 !!!

算术：add x5, x6, x7 : $x_5 = x_6 + x_7$

Data transfer：{ lw x5, 40(x6) : $x_5$ = Memory[$x_6$+40]; sw x5, 40(x6) : Memory[$x_6$+40] = $x_5$; lui x5, 0X12345 : $x_5$ = 0X12345000 }

logical：{ and x5, x6, x7 : x5 = x6 & x7，按位与; or 按位或; Xor 按位异或 → 异或：值相同为0，值不同为1 }

补：load 会符号扩展，Store 不用…

shift：{ sll x5, x6, x7 : $x_5 = x_6 << x_7$ 逻辑左移，空位填0.; srl：逻辑右移，空位填0（不进行符号扩展）; sra：算术右移，空位补原符号位. }

Branch if beqz：equal to zero

Conditional Branch { beq x5, x6, 100 : if x5=x6，则 $\xrightarrow{jump}$ pc+100; blt x5, x6, 100 : if x5<x6，则 $\xrightarrow{jump}$ pc+100 }

Unconditional Branch { jal x1, 100（相对跳转）: $x_1$ = pc+4, jump → pc+100; jalr x1, 100($x_5$)（绝对跳转）: $x_1$ = pc+4, jump → $x_5$+100 }

2进制表示：① unsigned，n bit，表示范围 [0, $2^n$-1]

② signed (complement)，nbit，范围 [$-2^{n-1}$, $2^{n-1}$-1]

对于补码，最高位符号位，1为负，0为正，其中 $\bar{x}+x$=11…1=-1

signed 计算：$x = -x_{n-1} \cdot 2^{n-1} + \sum_{0}^{n-2} x_k 2^k$

符号扩展：若unsigned，补0；若signed，补符号位

| | 31 | 25 24 | 20 19 | 15 14 | 12 11 | 7 6 | 0 |
|---|---|---|---|---|---|---|---|
| R | funct7 | rs2 | rs1 | funct3 | rd | opcode | |
| I | imm[11:0] | | rs1 | funct3 | rd | opcode | |
| S | imm[11:5] | rs2 | rs1 | funct3 | imm[4:0] | opcode | |
| (S)B | imm[12\|10:5] | rs2 | rs1 | funct3 | imm[4:1\|11] | opcode | |
| U | imm[31:12] | | | | rd | opcode | |
| (U)J | imm[20\|10:1\|11] | | imm[19:12] | | rd | opcode | |

opcode对应：

R：0110011

I：{ 0000011 lw; 0010011 addi; 1100111 jalr }

S：0100011

B：1100111

U：0110111

J：1101111

J/B默认imm[0]=0，不保存

*R：操作数均为Register

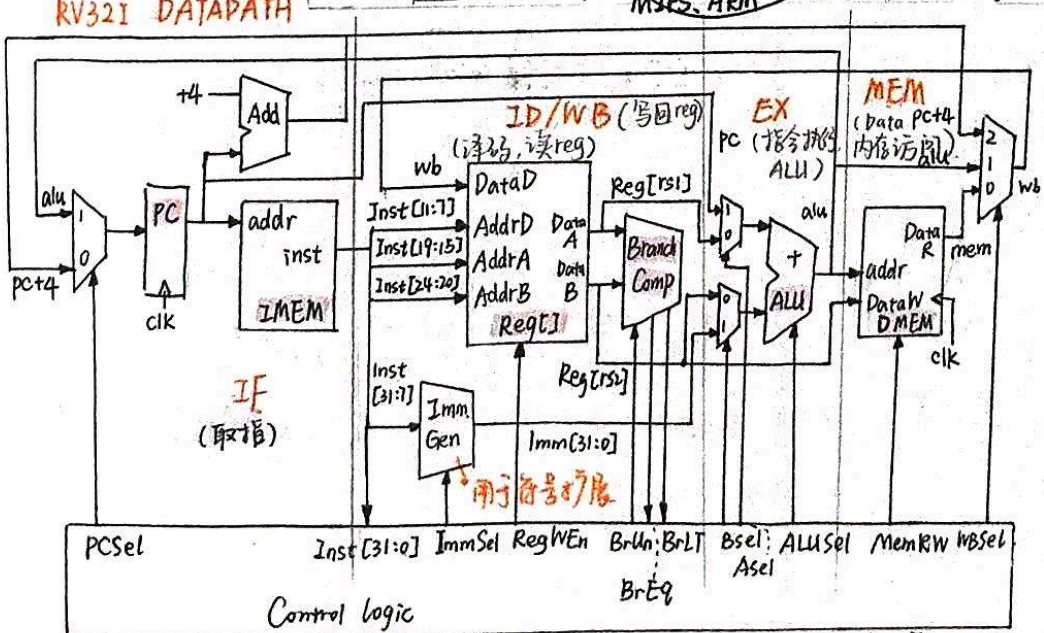
I：操作数含立即数.

SB不保存imm[0]，必=0.

R：add, and, sll　I：addi, lw, jalr

S：sw，B：beq, blt，U：lui，J：jal

RV32I DATAPATH




*栈和堆的开辟方向相反.

e.g. △栈先进后出!!!

addi sp, sp, -12（开辟空间）

sw x5, 8(sp)

sw x6, 4(sp) ⇐ 先存

sw x20, 0(sp) 高地址.

……

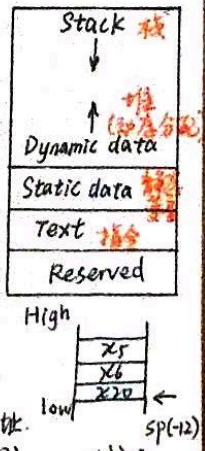
lw x20, 0(sp)

lw x6, 4(sp) ⇐ 先释

lw x5, 8(sp) 放低地址

addi sp, sp, 12（释放空间）

jalr x0, 0(x1)（返回原地址）




push：压栈 pop：弹栈

*critical path：用时最长的信号通路.

△注：single-cycle时，load指令需用到所有stage，用时max，可用于计算maximum clock frequency !!!

chapter 3

Binary normalized $\pm 1.xxx_2 \times 2^{yyy}$

Float Point 浮点数表示

浮点数标准(IEEE 754) { single: 32 bit / double: 64 bit

| S | exponent | Fraction |
|---|---|---|

符号位 single: 8, 32-1-8=23

正负1 double: 11, 64-1-11=52

$x=(-1)^S \times (1+\text{Fraction}) \times 2^{(\text{exponent}-\text{Bias})}$

若single, Bias $=2^{8-1}-1=127$

若double, Bias $=2^{11-1}-1=1023$

* Exponent 范围为 [000...1, 111...0], 排除全0和全1

则 single 范围 [1, 254], actual exponent [-126, 127]

double 范围 [1, 2046], actual exponent [-1022, 1023]

e.g. 将-0.75转为2进制single

$-0.75=(-1)^1 \times (1.1)_2 \times 2^{-1}$

则 ① S=1.

② exponent-Bias=-1

即 exponent=126=01111110$_2$

③ Fraction=100...0$_2$

则 Binary: 1 01111110 100...0$_2$

e.g. 将single转为十进制

Binary: 1 10000001 0100...0

① S=1

② exponent=129

则 exponent-Bias=2

③ Fraction=0100...0

十进制: $(-1)^1 \times (1.01)_2 \times 2^2=-5$

3条件

{ Fraction=全0, 无穷大

Fraction≠全0, NaN

注: ① 若exponent=全1.

② 浮点数运算交换律不成立.

③ 余数、被除数符号位应相同. ④ 右移1位=除$2^i$

只对无符号数成立! 有符号时, 逻辑或算术右移都不行.

EX前递条件: EX/MEM.RegWrite ①

EX/MEM.RegisterRD = ID/EX.RegisterRs1或2 且≠0 ②

MEM前递条件: 同上, 把EX/MEM替换为MEM/WB即可

I-cache miss rate 2%

D-cache ~4%

miss penalty=400 cycles

Base CPI=2. Load/store 36% of all instructions

Actual CPI=2+2+1.44=5.44

I: 0.02×100 D: 0.36×0.04×100

chapter 4

① Instruction count (指令数) 由ISA和compiler决定

② CPI 和 cycle time (平均时钟周期数、时钟周期) 由CPU hardware决定. IPC=1/CPI (Instructions per cycle)

关于target address计算: 实际PC=PC+保存的偏移量左移1位

Pipelining: IF、ID、EX、MEM、WB (MEM只有load/store会用到)

speedup 核心: 吞吐量增加, 但每条指令延迟时间: pipeline > nonpipeline

最理想情况(每stage的latency相同), 则(time between instructions)$_{pipeline}$ = (time between instructions)$_{nonpipeline}$ / stage numbers

即理想时, speedup=n ⇒ n级流水线

Hazards:

① Structure hazards 结构冒险(资源冲突)

解决方法: 针对memory型, 到不同cache取.

针对register型, 用double pumping. 上半周期写, 下半周期读.

例如 add $x_1, x_2, x_3$

sub, $x_4, x_1, x_5$

若无double pumping, 3个bubble

若有↑, 则2个bubble可解决.

add: IF → ID → EX → MEM → WB

sub: IF → ID → EX

② Data hazards 数据冒险 (寄存器读到旧值, 读未写入/下条已读)

依赖关系: 读后读、读后写、写后读、写后写 (只有写后读存在data hazards)

解决方法: (i) 一般情况, stall (插入bubble) 或 forwarding 可解决.

forwarding: IF→ID→EX→MEM→WB

(同上个例子) IF→ID→EX→MEM→WB

注: 转发应从上一阶段的结果转发, 否则流水线寄存器值可能不准确

(ii) load-use data hazards: 无法只用forwarding解决, 必须Do stall.

例 lw x1, 0(x2)

sub x4, x1, x5.

lw: IF→ID→EX→MEM→WB.

sub: IF→ID→EX→MEM→WB.

lw: IF→ID→EX→MEM→WB

IF→ID→EX→

③ Control hazard 控制冒险 (条件跳转, 提前取了非target address的指令).

解决方法: (i) stall (插入bubble). 若MEM后才判断出结果, 3个bubble; 若EX后..., 2个bubble.

RISC-V中在ID提前进行条件判断, 此时只需1个bubble. (Add hardware in ID stage)

(ii) 分支预测: 若猜测正确, 不需stall; 猜错与直接stall的时间一样, 须flush来清除错误.

静态预测 (固定). 动态预测 (基于历史行为判断) [1位Predictor用前1次信息, 2位predictor只在前2次都猜错时改变预测]

(iii) 其他方式: Delayed Branch 延时槽 补: data hazard 也可通过compiler调整代码顺序

chapter 5

局部性原则 (Principle of Locality) ①: 最近用过还会用 ②: 最近用过的旁边的, 可能会用

① 时间局部性 Temporal locality (e.g. 循环指令)

② 空间局部性 Spacial locality (e.g. 数组数据, 顺序指令访问)

Cache (SRAM) 访问速度↑成本↑. 主存 (DRAM) 访问速度↓成本↓

Cache中除了Data, 还有Valid、Tag.

| Tag | index | offset |
|---|---|---|

block address

offset bit $=\log_2$(Block size)

注: { Valid: 判断有效

tag: 判断是不是该block

offset: 用来确定是Block中的哪个字节

Tag = Address位数 − Index − offset, entry = Block数目.

$2^{\text{index bit}}$ = Block数目 / n, n为n-way组相联

若题目提供了Block的号, 则Index的值 = block号 / 组数 (取余)

相连度: ① 直接映射 ② N-way ③ Fully associative (miss rate↓)

成本低, 命中率高

N-way: 每组多路. 即每组中有多个block; 组: set, 总共多少组, $2^{\text{index bit}}$

Write Hit, 则可Write Back或Write through.

{ 写直达 (写通): 同时更新cache和memory, 但速度慢 (缓冲区 (可加buffer))

写回: 只更新cache, 置Dirty=1. 若脏读被替换, 再写回内存.

Write miss, 则有两种策略: write allocation / 写不分配.

{ 写分配: 从主RAM加载Block到cache, 在cache中修改 (总和写回策略)

写不分配: 不加载到cache, 直接写到主RAM中, 跳过cache.

Block Size Tradeoff: BS↑ Miss penalty↑ miss rate↓↑ AMAT↓↑

替换策略: LRU、RAndom (LRU效好, 但硬件复杂, 大容量可近似用random) FIFO (先出先出)

缺失类型: ① compulsory miss (first访问某block, 冷启动失效)

② capacity miss (容量不够, cache小, 无法包含所有block)

③ conflict miss (组相联/直接映射, 竞争同一个组).

average memory access time

AMAT = hit time + miss rate × miss penalty

若多级cache e.g. 2级 L1: 2ns (access time), missrate=20%

L2: 20ns, 25%, access time of MEM=80ns

则 AMAT=2+20%×(20+25%×80)=10ns

(注) 规律: 速度越快, 容量越小

CPI = Base CPI + miss rate × miss penalty

Miss penalty = Addr transfer + DRAM access + Data transfer

Memory stall cycles = $\frac{\text{Memory access}}{\text{program}} \times \text{missrate} \times \text{miss penalty} = \frac{\text{instructions}}{\text{Program}} \times \frac{\text{misses}}{\text{instruction}} \times \text{miss penalty}$

2-way, 4block cache

| index | valid | tag | data |
|---|---|---|---|
| 0 | | | |
| 1 | | | |

1个entry对应1个tag, entry=行数

异常 (exception) 由内部引起, interrupt (中断) 由外部引起

VM: (主memory和disk的层次关系, 理解为主memory为disk的cache.

① page table (全相连, 无tag, 用VPN查找!!!)

| VPN | Valid | Dirty | Ref | PPN |
|---|---|---|---|---|
| 0 | 1 | ⋮ | ⋮ | ⋮ |
| 1 | 0 | ⋮ | ⋮ | ⋮ |
| 2 | 0 | ⋮ | ⋮ | ⋮ |
| ⋮ | ⋮ | | | |

行数 $=2^{\text{VPN bit}}$

② TLB快表. (TLB是cache, 有tag位) 在CPU中.

| Valid | Dirty | Ref | Tag | PPN |
|---|---|---|---|---|
| | | | ⋮ | ⋮ |
| | | | | |
| ⋮ | | | | |

用tag查找!!!

③ 对应关系:

Virtual address 虚拟地址

| VPN | page offset |
|---|---|

translate.

| PPN | page offset |
|---|---|

physical address

公式: VPN = $\frac{\text{Virtual address}}{\text{page size}}$ (取整)

对于TLB: 全相连, Tag=VPN,

若n-way, Tag=VPN/组数 (取整)

index=VPN/组数 (取余)

offset $=\log_2$(page size)

例如: page size=4KiB. 4098 (Virtual addr) 的VPN=4098/4×$2^{10}$=1

① TLB hit

② 若TLB miss, 检查页表. 若页表中有对应转换, Valid=1, 则page hit

③ 若页表中无对应转换, 或Valid=0. page fault

*VM使用Write-Back, 因为Disk访问太慢

PTE=4B. cache line size

若VM中有 $2^n$ 个page, 则VPN=n bit. =Block size

the higher the memory bandwidth, the larger the cache block ✓

the shorter the memory latency, the smaller the cache block ✓

LRU can be improved by providing a little randomness ✓

the direct mapping is costlier than associated mapping ×

speed of operation increase, memory size increase ×

CPI of superscalar processors less than 1 ✓

the pipeline bubbling is a method to prevent data hazards and control hazards. ×

A program with no branches has high temporal locality ... ×

The entire page table is typically stored in physical memory ×

A cache that block size = word size of memory access instructions

There is no way to reduce compulsory misses ×

cache block size only affect miss latency ×

(但可影响miss rate)

Virtual-to-physical addr translation is on critical path of memory access ✓

spatical locality cannot exploit ✓

若无forwarding, 只用stall, 必须上一条写回WB, 下一条ID和WB同时

add x15, x12, x11

lw x13, 8(x15)

中间插入两个stall

*single cycle datapath 是指1个时钟周期完成1条指令 (无pipeline)

Amdahl's law: Sequential part can limit speed up.

e.g. loop processors, 90x speedup

Tnew = Tparallelizable/100 + Tsequential

Speedup = $\frac{1}{(1-F_{para})+F_{para}/100}$ = 90

解得 $F_{para}$=0.999. 即占99.9%

Parallelism

GPU-SIMT (单指令多线程) Thread-level

CPU-SIMD (单指令多数据) data-level parallel

multi-threading ⇒ thread有独立的PC, reg, stack memory, 共享code/static memory

cache size↑ index↑ tag↓

block size↑ miss penalty↑

spacial locality↑